

Blog

About Strapi

Company

Engineering

Ecosystem

Subscribe

ECOSYSTEM

• 9 MIN READ

Code-First vs Database-First: Which Approach Should You Use in 2025?

Beginner

Database

Backend Development



Paul Bratslavsky

• June 2, 2025

• *Updated on June 14, 2026*

Code-First vs Database-First: Which Approach Should You Use in 2025?



Paul Bratslavsky
Strapi Team



Choosing the right data access strategy directly affects development speed, maintainability, and integration with frontend frameworks. The two main options—code-first and database-first—each suit different project needs.

Code-first starts with your domain model. You define entities using classes, and the ORM generates the database schema from this structure. Database-first begins with an existing database. The ORM creates entity classes from the schema for use in your application.

In brief:

- Code-first gives you full control over the data model, aligns with domain-driven design, and works well for greenfield projects.
- Database-first is ideal when you're working with a legacy schema or need to prioritize database design and performance.
- Entity Framework supports both approaches and handles translation between your code and the database.
- Your choice should depend on your team's skills, project complexity, and long-term maintenance requirements.

Entity Framework: A Quick Overview

Entity Framework (EF) is an **object-relational mapper (ORM)** for .NET. It simplifies data access by letting you interact with databases using .NET objects, which removes the need to write low-level SQL or manage connections manually.

A key strength of EF is its support for multiple data modeling approaches:

- **Code-first:** You define the data model using C# classes. EF uses these to generate the database schema. This approach works well for domain-driven projects where code defines structure.
- **Database-first:** You start with an existing schema. EF reverse-engineers it into data model classes. This is a better fit for legacy systems or when the database design comes first.

Understanding the Code-First Approach

Code-first starts with your domain models, not the database. You define your data structures as classes, and the framework generates the database schema from them. This keeps business logic at the center and ensures your data model evolves directly from application code.

Workflow

The code-first process typically includes:

- Define entity classes and relationships.
- Register them with `DbContext`.
- Use migrations to generate or update the database schema.
- Run the application to apply schema changes.

This model-driven workflow gives you full control over your schema. As the application grows, migrations track database changes directly in version control.

Define your entities using C# or another .NET language. For example, a publishing system might include:

```
1 public class Author
2 {
3     public int AuthorId { get; set; }
4     public string Name { get; set; }
5     public DateTime DateOfBirth { get; set; }
6     public virtual ICollection<Book> Books { get; set; }
7 }
8
9 public class Book
10 {
11     public int BookId { get; set; }
12     public string Title { get; set; }
13     public DateTime PublicationDate { get; set; }
14     public int AuthorId { get; set; }
15     public virtual Author Author { get; set; }
16 }
```

These classes define entity properties and relationships. The `virtual` keyword enables lazy loading.

Code-first offers multiple ways to configure models:

1. **Data Annotations:** Attributes applied directly to model classes and properties:

```
1 public class Book
2 {
3     [Key]
4     public int BookId { get; set; }
5
6     [Required]
7     [StringLength(200)]
8     public string Title { get; set; }
9
10    [DataType(DataType.Date)]
11    public DateTime PublicationDate { get; set; }
12
13    [ForeignKey("Author")]
14    public int AuthorId { get; set; }
```

```
15
16     public virtual Author Author { get; set; }
17 }
```

1. **Fluent API:** For more complex configurations, the Fluent API provides a code-based approach:

```
1  protected override void OnModelCreating(ModelBuilder modelBuilder)
2  {
3      modelBuilder.Entity<Book>()
4          .HasKey(b => b.BookId);
5
6      modelBuilder.Entity<Book>()
7          .Property(b => b.Title)
8          .IsRequired()
9          .HasMaxLength(200);
10
11     modelBuilder.Entity<Book>()
12         .HasOne(b => b.Author)
13         .WithMany(a => a.Books)
```



Advantages of Code-First

- **Full Developer Control:** Use inheritance, polymorphism, and encapsulation in your models.
- **Domain-Driven Design Alignment:** Your domain model shapes the schema, not the reverse.
- **Single Source of Truth:** Code defines both the data structure and the business logic.
- **Simplified Workflow:** You work entirely in code to reduce context switching.
- **Faster Iteration:** Schema updates come directly from code changes and migrations.
- **Test-Friendly:** Easily use in-memory databases since the schema is generated from code.

Limitations to Consider

- **Learning Curve:** Developers familiar with SQL-first workflows may need time to adjust.
- **Migration Complexity:** Evolving schemas with existing data requires careful migration planning.
- **Performance Tuning Limits:** Low-level DB optimizations may be harder to apply due to ORM abstraction.

When to Use Code-First

Code-first is a strong choice when:

- Starting a new project where the data model is still evolving
- Prioritizing developer control over database schema design
- Working in agile environments that require frequent, rapid iteration
- The application logic defines the structure and relationships of data
- Teams want to manage database changes through version-controlled code (e.g., using migrations)

Understanding the Database-First Approach

Database-first begins with an existing database schema and generates entity classes from it. This approach emphasizes database design and optimization, which is ideal for data-heavy applications or projects built on legacy systems.

Workflow

With database-first, the typical steps include:

- Create or use an existing database with tables, constraints, and relationships
- Use scaffolding tools to generate `DbContext` and entity classes
- Customize the generated code if needed
- Regenerate or sync the model when the schema changes

This approach is especially effective for integrating new applications with established databases. It avoids manual migration scripting and works directly with existing schema designs

Advantages of Database-First

1. **Seamless Integration:** Connects directly to legacy databases or shared schemas without changing the existing structure.
2. **Automatic Code Generation:** Tools reverse-engineer schema into a strongly typed model, reducing manual coding and errors.
3. **Use of Proven Designs:** Leverages existing indexes, constraints, triggers, and normalization from optimized databases.
4. **Support for Complex Schemas:** Handles stored procedures, views, and intricate relational structures more easily than code-first.

For a deeper understanding of different [relational database features](#), exploring systems like PostgreSQL, MariaDB, MySQL, and SQLite can provide insights into how these databases can influence your database-first approach.

Limitations to Consider

- **Reduced Flexibility:** Domain model changes require corresponding database changes first, slowing iterations.
- **Code Regeneration Risk:** Schema updates trigger regeneration, which can overwrite custom logic or break code unless manually adjusted.
- **Drift Between Layers:** Without disciplined syncing, the database and codebase can fall out of sync.
- **Mapping Limitations:** Some object-oriented patterns, such as inheritance, don't translate cleanly to relational models.

When to Use Database-First

Database-first is a strong choice when:

- Working with large or critical legacy databases that should remain unchanged

- Modernizing the frontend while keeping the existing backend
- The database is the primary source of truth for business logic or compliance
- Multiple systems share the same data layer

Code-First vs Database-First: Making the Right Choice for Your Project

Choosing between code-first and database-first approaches depends on your project's context. Use the following framework to evaluate which approach is best.

Project Type

- **New Projects:** Code-first gives you flexibility. You can design models in code and evolve the schema as the application grows. It aligns well with agile workflows and domain-driven design.
- **Existing Systems:** Database-first is better suited for legacy databases or complex schemas. It reduces manual mapping and integrates with existing systems immediately.

Team Skills and Workflow

- Choose code-first if your team is stronger in object-oriented programming and less experienced with database administration.
- Choose database-first if your team includes DBAs or the project depends heavily on stored procedures, triggers, or database-specific logic.

Database Complexity

- Use database-first for applications with complex relational structures, stored procedures, and custom views.
- Use code-first for applications where the schema is relatively simple and tightly aligned with business logic.

For deeper schema planning, explore different [SQL relationship types](#) and review the [SQL vs NoSQL comparison](#) to assess which database type aligns with your architecture.

Future Maintenance and Evolution

- Choose code-first for frequent schema changes. It supports versioning through Entity Framework Migrations, making it easier to track and apply changes over time.
- Choose database-first if your schema will remain stable or if DBAs need to maintain strict control over the structure.

Also, consider [how different teams will collaborate](#). Code-first favors developer-driven workflows, while database-first can centralize control with data architects.

Business and Regulatory Requirements

- Database-first may be more appropriate if your organization requires strict audit trails, compliance controls, or granular database access.
- Ensure you factor in stakeholder or regulatory input that could shape how your data model is created and maintained.

How Strapi Complements Code-First and Database-First Approaches

[Strapi](#), as a headless CMS, supports both code-first and database-first approaches by enhancing its strengths and reducing friction in content and schema management.

For Code-First Developers

Strapi's visual content modeling interface mirrors how you define entities in code. With the [Content-Type Builder](#), you can:

- Visually prototype data models in a familiar object-oriented structure
- Define relationships, fields, and constraints without writing schema migrations

- Quickly translate these models into backend APIs (REST or GraphQL)

This makes Strapi an effective companion for developer-first workflows, allowing rapid iteration without giving up control over the data model.

For Database-First Projects

Strapi integrates with existing databases through custom configurations. If you're working with a legacy schema:

- Strapi can expose the data via modern **REST and GraphQL APIs**
- You avoid rewriting your schema while adding structured content management
- It supports gradual modernization without disrupting the underlying database

This lets database-centric teams layer in Strapi's CMS capabilities without large-scale refactoring.

Shared Benefits for Both Approaches

Regardless of your approach, Strapi offers:

- **Role-based access control (RBAC)** for secure, scoped access to content
- **Webhooks and lifecycle hooks** for event-driven integrations
- **Extensible APIs** that standardize how frontend teams interact with backend data

Strapi's plugin ecosystem adds even more power with tools for:

- Database introspection and visualization
- Advanced querying and filtering
- Data transformation workflows

Choose Your Path Wisely

Choosing between code-first vs database-first depends on your project's specific needs. Both approaches offer strengths—your decision should reflect team skills, existing infrastructure, and long-term goals

- Code-First works best for new projects. It gives developers control over the data model, enables faster iteration, and supports domain-driven design.
- Database-first is ideal for working with established schemas, especially in enterprise environments where data structures are already optimized and in use.

Understanding both gives you the flexibility to adapt. Some projects start with one approach and shift to another as complexity increases. Having the ability to work with either model strengthens your ability to choose the right tool for the job.

If you're using a [headless CMS](#), this headless CMS overview can help clarify how content modeling complements your data strategy. Tools like **Strapi** provide visual modeling with code-first flexibility and support existing databases for database-first workflows.

With **Strapi v5**, you get performance improvements, improved DX, and powerful content modeling tools. The latest version works seamlessly with your preferred data modeling approach. Explore the [Strapi v5 documentation](#) to get started.

To go further:

- Learn the essentials of [content modeling](#).
- Compare [headless vs traditional CMS](#) to evaluate your platform options.
- [Explore Strapi Cloud](#) for a fully managed solution that lets you deploy and scale your Strapi projects with zero infrastructure setup.

Whether you're starting from code or integrating with a legacy database, Strapi Cloud gives you a fast, flexible way to build, manage, and scale your content layer.

